



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

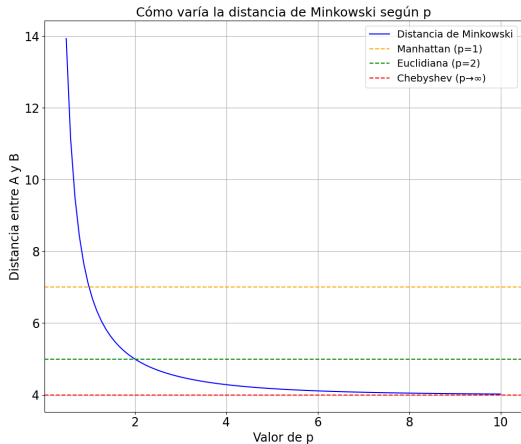
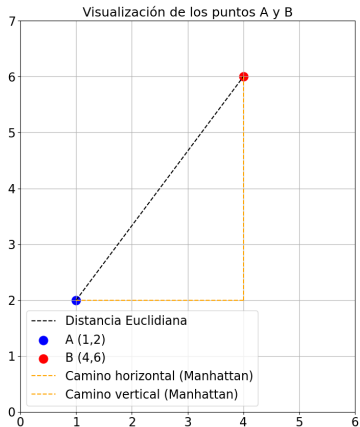
K-nearest Neighbors and K-means Algorithms

EIN087B - Ciencia de Datos
Paralelo 701

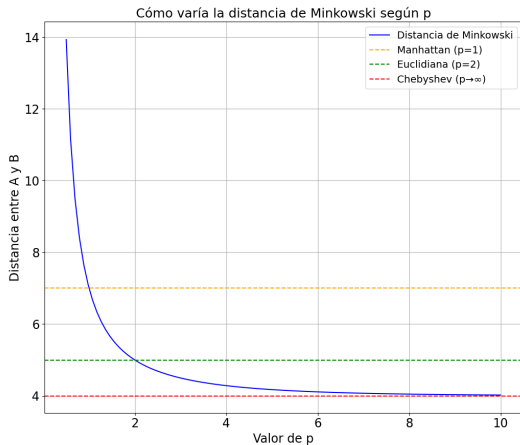
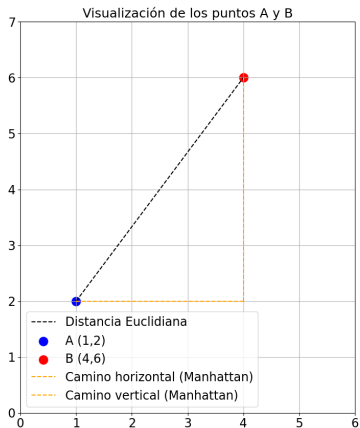
Profesor: Jorge Portilla G.
`jorge.portilla@usm.cl`

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile

Distancia de Minkowski

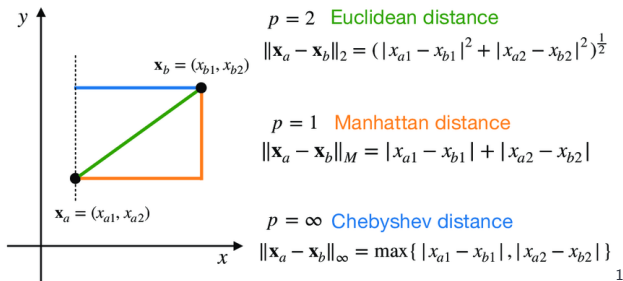


Distancia de Minkowski



$$d_{\text{minkowski}} = (|x_2 - x_1|^p + |y_2 - y_1|^p)^{\frac{1}{p}} \quad (1)$$

Distancia de Minkowski



1

Cuando el parámetro p tiende a infinito, se define como el máximo de las diferencias absolutas entre las coordenadas de dos puntos.

¹Fu, C., Yang, J. (2021). Granular classification for imbalanced datasets: a minkowski distance-based method. Algorithms, 14(2), 54.

Distancia de Minkowski:

es una generalización de las distancias Euclidiana y Manhattan. Se calcula la distancia de Minkowski entre los puntos P_1 y P_2 con un parámetro p utilizando la expresión:

$$d_{\text{minkowski}} = (|x_2 - x_1|^p + |y_2 - y_1|^p)^{\frac{1}{p}} \quad (2)$$

Dependiendo del valor de $\underline{p}$:

- Si $p = 1$, se obtiene la distancia Manhattan.
- Si $p = 2$, se obtiene la distancia Euclidiana.
- Si $p = \infty$, se obtiene la distancia Chebyshev.

► **Caso cuando $p \rightarrow \infty$:**

- Considere $a = |x_2 - x_1|$ y $b = |y_2 - y_1|$.
- Si $p \rightarrow \infty$, entonces $a^p + b^p$ estará dominado por el mayor de los dos términos.
- Suponiendo que $a \geq b$, entonces $a^p + b^p \approx a^p$ cuando p es muy grande.

► **Límite:**

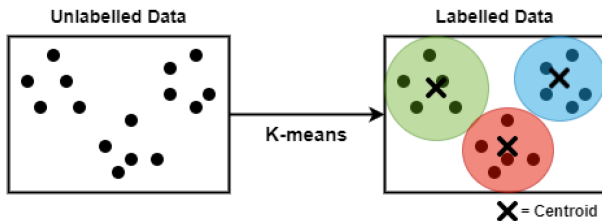
$$d_\infty = \lim_{p \rightarrow \infty} (a^p + b^p)^{\frac{1}{p}} \approx \lim_{p \rightarrow \infty} (a^p)^{\frac{1}{p}} = a$$

► **Resultado:**

- Cuando $p \rightarrow \infty$, $d_{\text{minkowski}}$ se convierte en $\text{máx}(a, b)$, que es la **Distancia de Chebyshev**.

K-means

K-means Clustering



- ▶ **K-means clustering** es una técnica en la que colocamos cada observación en un conjunto de datos en uno de K clusters.
- ▶ El objetivo es tener K clusters agrupando las observaciones similares entre sí.
- ▶ En la práctica, utilizamos los siguientes pasos para realizar K-means clustering:

Pasos para realizar K-means clustering

- ▶ **Clustering** es el proceso de dividir todo el conjunto de datos en grupos basados en los patrones observados en él.
- ▶ El objetivo principal del algoritmo K-Means es minimizar la suma de las distancias entre los puntos y sus respectivos centroides de clúster.

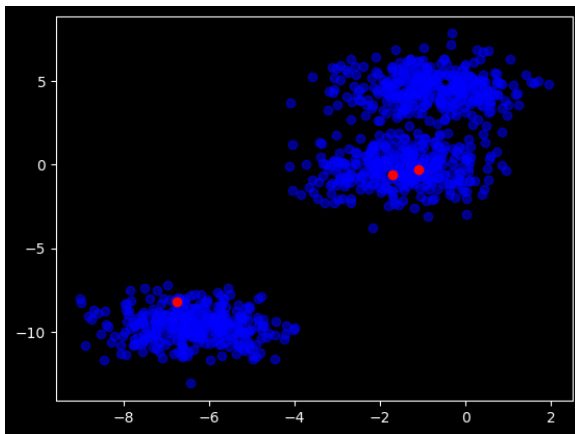
Pasos del Algoritmo K-Means:

1. Elegir el número de clústeres k .
2. Seleccionar k puntos aleatorios del conjunto de datos como centroides.
3. Asignar todos los puntos al centroide del clúster más cercano.
4. Recalcular los centroides de los clústeres recién formados.
5. Repetir los pasos 3 y 4 hasta que los centroides no cambien.

K-means

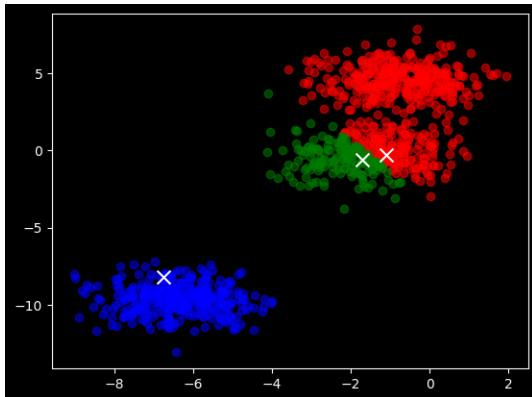
Dado un dataset sintético creado con `make_blobs` que contiene 1000 ejemplos, 2 características y clusters.

1. **Paso 1:** K igual a 3.
2. **Paso 2:** Inicializar los centroides aleatoriamente².



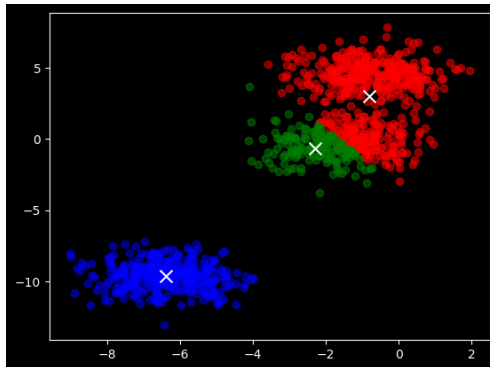
²Selecciona 3 ejemplos de los datos.

3. **Paso 3:** Asignar cada punto de datos al cluster más cercano.
- ▶ Se inicializa una lista de clusters vacíos y luego, para cada punto en el conjunto de datos **X**.
 - ▶ Se calcula la distancia euclidiana desde ese punto hasta cada uno de los centroides

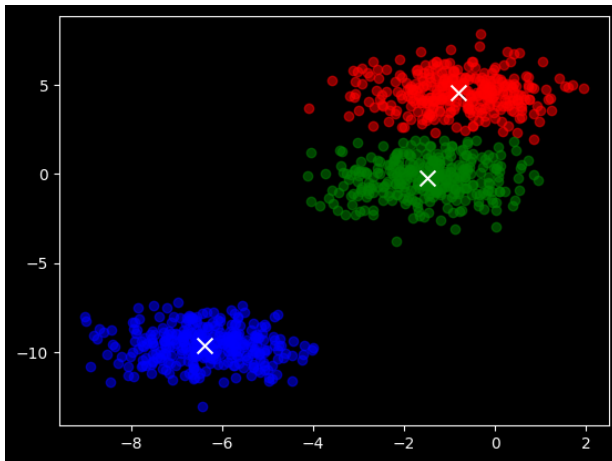


4. Paso 4: Asignar cada punto de datos al cluster más cercano.

- ▶ Se recalcula los centroides de cada cluster z_3 tomando la media de los puntos asignados a ese cluster.
- ▶ Se inicializa una matriz de centroides y se calcula la media de los puntos en ese cluster, actualizando la posición del centroide.
- ▶ Se repite hasta que los centroides se estabilizan, permitiendo que los clusters converjan a su forma final.



5. **Paso 5:** Repetir pasos 3 y 4 hasta que los centroides no cambien.



¿Qué es `kmeans.inertia_`?

- ▶ `kmeans.inertia_` es un atributo de un objeto K-Means en `scikit-learn` que representa la suma de las distancias al cuadrado del modelo de clustering K-means.
- ▶ La inercia es la suma de los cuadrados de las distancias de cada punto de datos a su centroide más cercano.
- ▶ Es una medida de qué tan bien se agrupan los puntos dentro de sus respectivos clústeres.

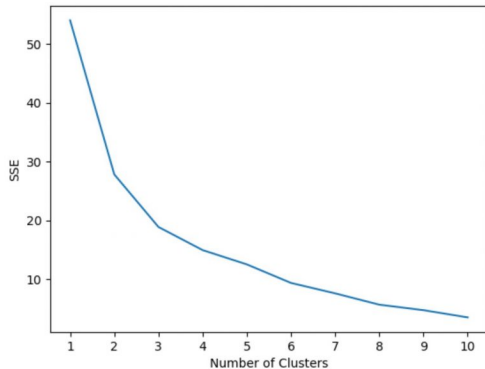
La suma de distancias al cuadrado se define matemáticamente como:

$$\text{SSE} = \sum_{i=1}^n \min_{c \in \{1, \dots, K\}} \|x_i - \mu_c\|^2$$

donde:

- ▶ n es el número de puntos de datos.
- ▶ K es el número de clusters.
- ▶ x_i es el vector de características del punto de datos i .
- ▶ μ_c es el centroide del cluster c .
- ▶ $\|x_i - \mu_c\|^2$ es la distancia al cuadrado entre el punto de datos i y el centroide c .

- ▶ **Valores Bajos de Inercia:** Indican que los puntos dentro de un cluster están cerca de su centroide.
- ▶ **Valores Altos de Inercia:** Pueden indicar que los puntos no están bien agrupados o que hay mucha dispersión dentro de los clusters.
- ▶ Al evaluar diferentes valores de K (número de clusters), los modelos con menor inercia suelen considerarse mejores, ya que implican clusters más compactos.



- El **método del codo** es una técnica común para **identificar el K óptimo**, buscando el punto en el que la disminución de la inercia deja de ser tan pronunciada.
- Sum of Squared Errors (SSE).

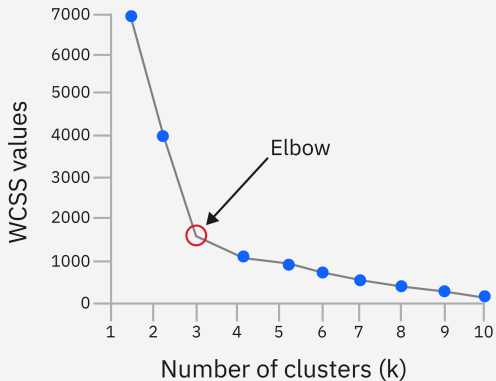


Figure 1: K-means clustering elbow

- **Problema Clásico en K-means:** La calidad del clustering en K-means depende fuertemente de la inicialización de los centroides. Una mala inicialización puede conducir a soluciones subóptimas o a la convergencia en mínimos locales.
- **K-means++**
 - Introducido para mejorar la robustez del K-means.
 - Elegir centroides iniciales que estén óptimamente distanciados entre sí.
 - Selecciona el primer centroide de manera aleatoria.
 - Para cada punto restante, selecciona un nuevo centroide con una probabilidad proporcional al cuadrado de la distancia mínima a cualquier centroide ya elegido.
 - Repite hasta que se hayan seleccionado K centroides.

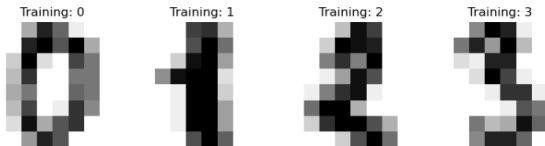
En un conjunto de datos con puntos muy dispersos, k-means++ se enfoca en que los primeros centroides estén en diferentes partes del espacio de datos, mientras que el k-means puede seleccionar varios centroides muy cercanos entre sí, lo que puede llevar a un rendimiento deficiente.

- Después de seleccionar el primer centroide al azar, los siguientes centroides se seleccionan utilizando una probabilidad proporcional.
- Para cada punto x , se calcula la distancia $D(x)$ al centroide más cercano ya seleccionado.
- El siguiente centroide se selecciona con probabilidad proporcional a $D(x)^2$, es decir:

$$P(x) = \frac{D(x)^2}{\sum_{x \in \text{datos}} D(x)^2}$$

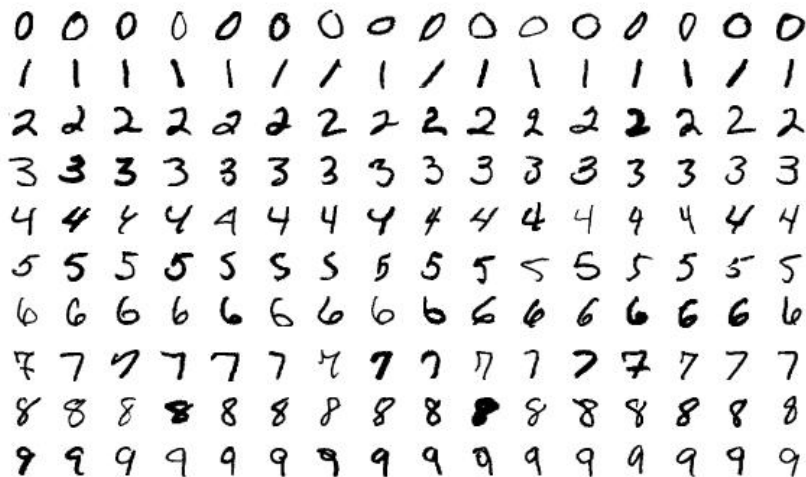
- Esto significa que los puntos más alejados de los centroides seleccionados tienen una mayor probabilidad de ser elegidos como nuevos centroides.

Ejemplo de K-means - Recognizing hand-written digits



- ▶ **Nombre:** Dígitos Escritos a Mano (Handwritten Digits)
- ▶ **Imágenes:** 1797 imágenes de 8x8 píxeles
- ▶ **Clases:** 10 (dígitos del 0 al 9)
- ▶ **Etiquetas:** Cada imagen está etiquetada con el dígito correspondiente
- ▶ **Objetivo:** Clasificar correctamente las imágenes en una de las 10 categorías
- ▶ **Desafíos:** Variabilidad en la escritura a mano y similitud entre ciertos dígitos

Recognizing hand-written digits



- ▶ Compararemos tres enfoques:
 - **Una inicialización usando k-means++.** Este método es estocástico, y ejecutaremos la inicialización 4 veces.
 - **Una inicialización aleatoria.** Este método también es estocástico, y ejecutaremos la inicialización 4 veces.
 - **Una inicialización basada en una proyección PCA.** Usaremos los componentes de la PCA para inicializar KMeans. Este método es determinista y una sola inicialización es suficiente.
- ▶ En este ejemplo, comparamos las diferentes estrategias de inicialización para K-means en términos de tiempo de ejecución y calidad de los resultados.

```
1 from sklearn.cluster import KMeans
2 from sklearn.decomposition import PCA
3 kmeans = KMeans(init="k-means++", n_clusters=n_digits,
4                 n_init=4, random_state=0)
5 bench_k_means(kmeans=kmeans, name="k-means++", data=data,
6               labels=labels)
7
8 kmeans = KMeans(init="random", n_clusters=n_digits,
9                 n_init=4, random_state=0)
10 bench_k_means(kmeans=kmeans, name="random", data=data,
11               labels=labels)
12
13 pca = PCA(n_components=n_digits).fit(data)
14 kmeans = KMeans(init=pca.components_, n_clusters=n_digits,
15                 n_init=1)
16 bench_k_means(kmeans=kmeans, name="PCA-based", data=data,
17               labels=labels)
```

***n_init**: number of times the k-means algorithm is run with different centroid seeds.

Ejemplo de K-means - Recognizing hand-written digits

init	time	inertia	homo	compl	v-meas	ARI	AMI	silhouette
k-means++	0.050s	69545	0.598	0.645	0.621	0.469	0.617	0.152
random	0.064s	69735	0.681	0.723	0.701	0.574	0.698	0.170
PCA-based	0.017s	69513	0.600	0.647	0.622	0.468	0.618	0.162

- **time:** El tiempo que tarda en ejecutarse el algoritmo.
- **inertia:** Suma de las distancias al cuadrado entre los puntos de datos y sus centroides.
- **homo (homogeneidad):** Mide qué tan homogéneos son los clusters; es decir, qué tan bien un solo cluster contiene únicamente puntos de una sola clase.
- **compl (completitud):** Mide qué tan completos son los clusters; es decir, qué tan bien todos los puntos de una clase se encuentran dentro del mismo cluster.
- **v-meas:** La media armónica de homogeneidad y completitud, una medida equilibrada entre ambas.

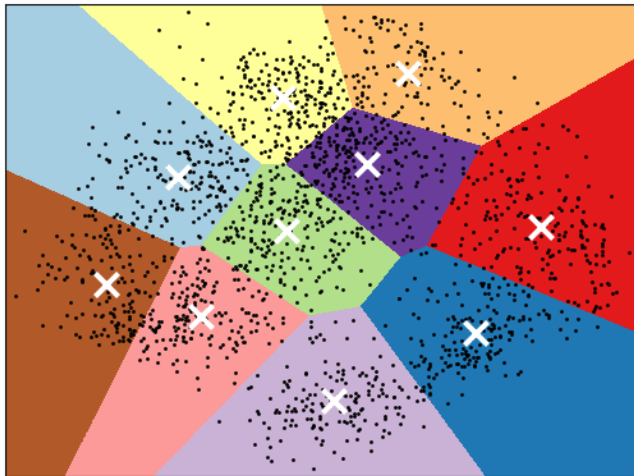
Ejemplo de K-means - Recognizing hand-written digits

init	time	inertia	homo	compl	v-meas	ARI	AMI	silhouette
k-means++	0.050s	69545	0.598	0.645	0.621	0.469	0.617	0.152
random	0.064s	69735	0.681	0.723	0.701	0.574	0.698	0.170
PCA-based	0.017s	69513	0.600	0.647	0.622	0.468	0.618	0.162

- ▶ **v-meas:** La media armónica de homogeneidad y completitud, una medida equilibrada entre ambas.
- ▶ **ARI (Adjusted Rand Index):** Una medida que compara la similitud entre las agrupaciones generadas por el algoritmo y una agrupación de referencia, ajustada por la probabilidad de coincidencias aleatorias.
- ▶ **AMI (Adjusted Mutual Information):** Indica qué tan similares son las agrupaciones producidas por el algoritmo a una clasificación esperada, ajustada por la probabilidad de coincidencias aleatorias.
- ▶ **silhouette:** Un índice que mide qué tan similares son los puntos a su propio cluster en comparación con otros clusters, con valores entre -1 y 1 (cercano a 1 es mejor).

Ejemplo de K-means - Recognizing hand-written digits

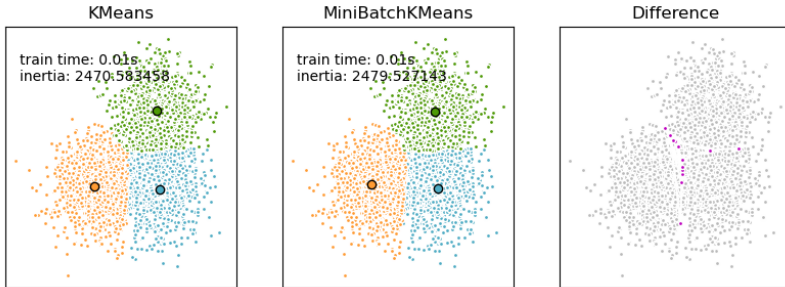
K-means clustering on the digits dataset (PCA-reduced data)
Centroids are marked with white cross



A demo of K-Means clustering on the handwritten digits data

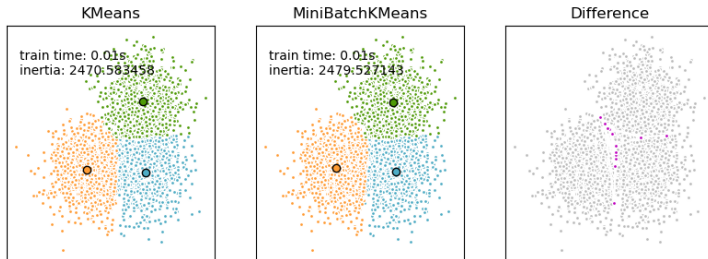
- ▶ K-means estándar puede ser computacionalmente costoso y lento en datasets grandes debido a la necesidad de recalcular los centroides en cada iteración sobre todos los datos.
- ▶ **Método:**
 - En lugar de usar el conjunto de datos completo en cada iteración, MiniBatchKMeans toma mini-batches de datos de forma aleatoria.
 - Actualiza los centroides únicamente a partir del mini-lote actual, lo que reduce drásticamente el tiempo de cálculo.

K-means vs. MiniBatchMeans



- ▶ **MiniBatchKMeans** es una variante del algoritmo K-Means que utiliza mini-batches para reducir el tiempo de cálculo.
- ▶ Aunque reduce el tiempo de cálculo, sigue intentando optimizar la misma función objetivo que el K-Means estándar.
- ▶ **Mini-batches:** Son subconjuntos de los datos de entrada, muestreados aleatoriamente en cada iteración de entrenamiento.
- ▶ **Ventaja:** Los mini-batches reducen drásticamente la cantidad de cálculo necesario para converger a una solución local.
- ▶ **Criterio de parada:** Los pasos se repiten hasta que se alcanza la convergencia o se alcanza un número predeterminado de iteraciones.

K-means vs. MiniBatchMeans (2)



- ▶ Por lo general, MiniBatchKMeans produce resultados de menor desempeño que los del algoritmo estándar.
- ▶ Se extraen muestras aleatorias del dataset (mini-batch).
- ▶ Estas muestras se asignan al centroide más cercano.
- ▶ Los centroides se actualizan, pero, a diferencia de K-Means, se actualizan por muestra.
- ▶ Para cada muestra del mini-batch, el centroide asignado se actualiza tomando el promedio acumulado de la muestra y de todas las muestras anteriores asignadas a ese centroide. Esto reduce la tasa de cambio del centroide con el tiempo.
- ▶ **Criterio de parada:** Los pasos se repiten hasta que se alcanza la convergencia o se alcanza un número predeterminado de iteraciones.

Actualización de centroides en MiniBatch K-Means

La actualización de centroides en MiniBatchMeans se realiza utilizando la siguiente fórmula:

$$C_{t+1} = C_t + \frac{1}{N_i}(x_i - C_t)$$

- ▶ x_i : Es el punto de datos del mini-batch actual asignado al centroide. Representa un vector de características del conjunto de datos.
- ▶ C_t : Es la posición actual del centroide en la iteración t .
- ▶ C_{t+1} : Es la nueva posición del centroide después de la actualización en la iteración $t + 1$.
- ▶ N_i : Es el número de puntos asignados al centroide C hasta la iteración i .

Actualización de centroides en MiniBatch K-Means

La actualización de centroides en MiniBatchMeans se realiza utilizando la siguiente fórmula:

$$C_{t+1} = C_t + \frac{1}{N_i}(x_i - C_t)$$

- ▶ x_i : Es el punto de datos del mini-batch actual asignado al centroide. Representa un vector de características del conjunto de datos.
- ▶ C_t : Es la posición actual del centroide en la iteración t .
- ▶ C_{t+1} : Es la nueva posición del centroide después de la actualización en la iteración $t + 1$.
- ▶ N_i : Es el número de puntos asignados al centroide C hasta la iteración i .
- ▶ Se ajusta el centroide C_t a partir de la información del punto x_i del mini-batch, moviéndolo ligeramente hacia x_i para que el centroide se acerque más a los puntos asignados a él.
- ▶ Este ajuste es incremental, lo que significa que el cambio en el centroide se vuelve más pequeño a medida que se procesan más puntos, estabilizando la posición del centroide con el tiempo.

K-Nearest Neighbors

K-Nearest Neighbors (KNN) algorithm

- ▶ El algoritmo KNN es uno de los algoritmos clásicos de aprendizaje automático supervisado.
- ▶ KNN es capaz de clasificar tanto en binario como en multiclase.
- ▶ Algoritmo de clasificación no paramétrico. No paramétrico por naturaleza, KNN también puede utilizarse como algoritmo de regresión.

- ▶ KNN es un algoritmo no paramétrico. Por lo tanto, el entrenamiento de un clasificador KNN no requiere recurrir al enfoque más tradicional de iterar sobre los datos de entrenamiento durante varias épocas para optimizar un conjunto de parámetros.
- ▶ El entrenamiento de KNN implica almacenar todas las instancias de entrenamiento en la memoria del equipo simultáneamente.
- ▶ Etapa de inferencia: el modelo compara estos nuevos datos con las instancias de entrenamiento existentes.

- ▶ **Cálculo de la distancia:** El modelo KNN calcula la distancia³ entre el nuevo punto de datos y todos los puntos de datos en el conjunto de datos de entrenamiento.
- ▶ **Selección de vecinos más cercanos:** El algoritmo selecciona un número k de puntos de datos de entrenamiento más cercanos al nuevo punto de datos.
- ▶ **Asignación de la clase:** El algoritmo asigna al nuevo punto de datos la clase que aparece con mayor frecuencia entre estos k vecinos.

³Distancia euclidiana

- ▶ **Objetivo de KNN:** Predecir respuestas cualitativas (clasificación) a partir de la proximidad a los puntos de entrenamiento conocidos.
- ▶ El clasificador KNN intenta aproximar la distribución condicional de Y dado X sin asumir un modelo paramétrico
- ▶ **Ventajas:**
 - No requiere suposiciones sobre la distribución de los datos.
 - Es simple y fácil de entender.
- ▶ **Desventajas:**
 - Sensible al valor de K y a la métrica de distancia utilizada.
 - Computacionalmente costoso para grandes datasets.

Definición de KNN:

- ▶ Dado un entero positivo K y una observación de prueba x_0 , el clasificador KNN identifica los K puntos más cercanos en los datos de entrenamiento.
- ▶ Estos puntos más cercanos se representan como $\mathcal{N}_0$.

⁴Métricas de distancia

Definición de KNN:

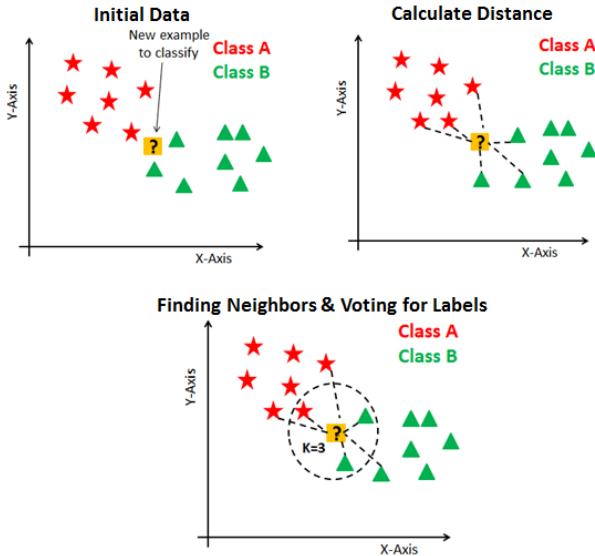
- ▶ Dado un entero positivo K y una observación de prueba x_0 , el clasificador KNN identifica los K puntos más cercanos en los datos de entrenamiento.
- ▶ Estos puntos más cercanos se representan como $\mathcal{N}_0$.
- ▶ La probabilidad condicional estimada para la clase j se calcula como la fracción de puntos en $\mathcal{N}_0$ cuyas respuestas corresponden a la clase j :

$$\Pr(Y = j \mid X = x_0) = \frac{1}{K} \sum_{i \in \mathcal{N}_0} I(y_i = j).$$

- ▶ Aquí, $\mathcal{N}_0$ representa el conjunto de los K puntos más cercanos a la observación de prueba x_0 , e $I(y_i = j)$ es una función indicadora⁴ que es 1 si $y_i = j$ y 0 en caso contrario.
- ▶ **Clasificación:** KNN clasifica la observación de prueba x_0 a la clase con la mayor probabilidad estimada, de acuerdo con la ecuación anterior.

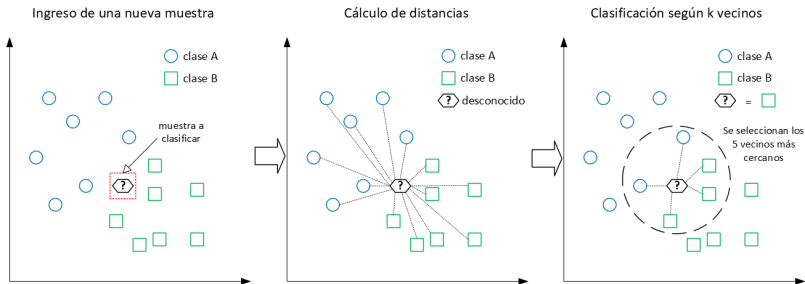
⁴Métricas de distancia

Ejemplo de KNN Classifier

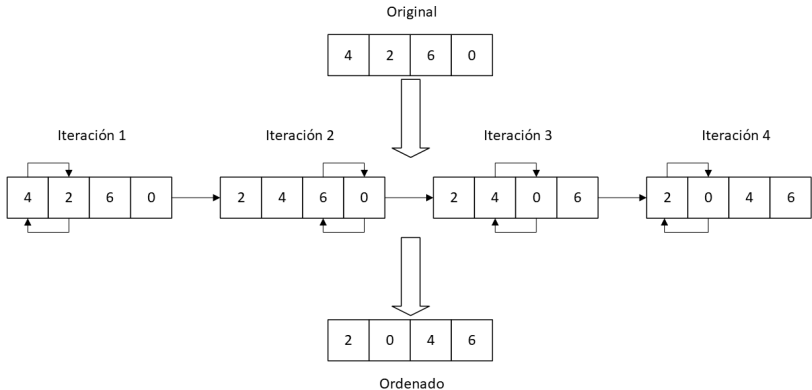


Para visualizar problemas con más de 2 o 3 dimensiones, se recomienda usar PCA.

Funcionamiento del algoritmo k-NN

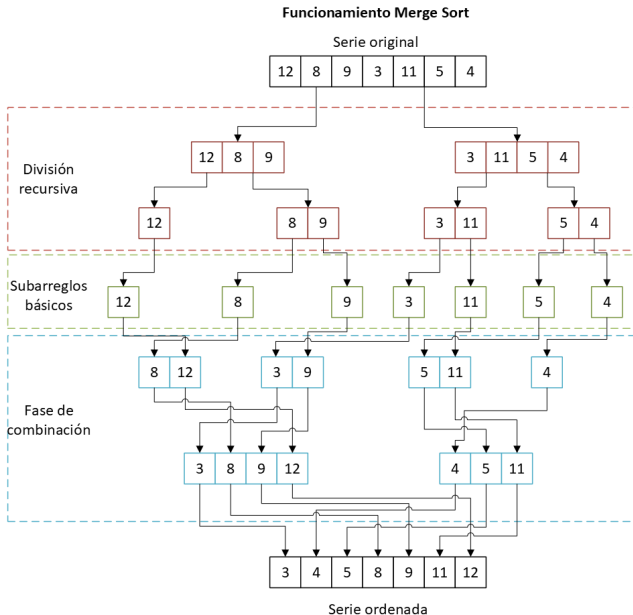


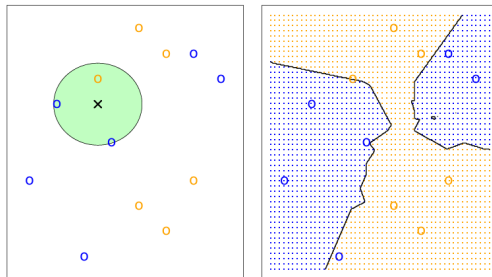
Funcionamiento Bubble Sort



⁵Diego Hernán Hidalgo Contreras, Diseño e implementación del algoritmo K-Nearest Neighbors en FPGA para clasificación binaria (Tesis de Grado, Ingeniería de Ejecución en Control e Instrumentación Industrial, UTFSM, 2025)

Merge sort





- El enfoque KNN, utilizando $K=3$, se ilustra en una situación simple con seis observaciones azules y seis observaciones naranjas.
- **Izquierda:** Se muestra una observación de prueba, donde se desea predecir una etiqueta de clase. Se identifican los tres puntos más cercanos a la observación de prueba, y se predice que la observación de prueba pertenece a la clase que ocurre con mayor frecuencia (azul).
- **Derecha:** Se muestra la frontera de decisión del KNN para este ejemplo en negro.

- ▶ `K-Neighbors Classifier` maneja un parámetro para identificar la métrica de distancia, llamado *metric*, el cual por defecto tiene como valor la distancia de **Minkowski**.
- ▶ `K-NeighborsClassifier` maneja un parámetro **p** para definir la **potencia** aplicada a la métrica distancia de Minkowski, siendo su valor por defecto $p = 2$.
- ▶ ¿Qué valor debíamos configurar para p ?

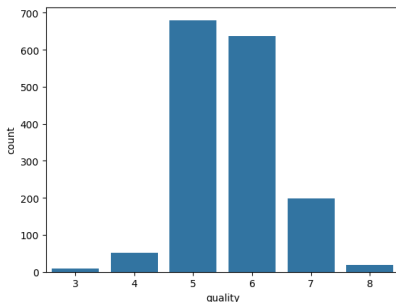
- ▶ UCI Red Wine Dataset: Contiene 1,599 muestras de vino tinto.
- ▶ El objetivo es predecir la **calidad del vino**, que es una variable ordinal en una escala de 0 a 10.
- ▶ **Variables del Dataset:**
 - **Fixed acidity:** Acidez fija (no volátil).
 - **Volatile acidity:** Acidez volátil (afecta al aroma).
 - **Citric acid:** Ácido cítrico (contribuye a la frescura).
 - **Residual sugar:** Azúcar residual (después de la fermentación).
 - **Chlorides:** Nivel de sal.
 - **Free sulfur dioxide:** Sulfuro de dióxido libre (conservante).
 - **Total sulfur dioxide:** Dióxido de azufre total.
 - **Density:** Densidad del vino.
 - **pH:** Mide la acidez.
 - **Sulphates:** Sulfatos (conservante).
 - **Alcohol:** Contenido de alcohol en el vino.

Ejemplo de KNN Classifier

- Para prevenir inestabilidades en el proceso de entrenamiento, estandarizamos el tensor de entrada $\mathbf{X}$ y el tensor de salida $\mathbf{Y}$ que contienen todos los pares de imágenes de entrada-salida de entrenamiento:

$$\bar{\mathbf{X}} = \frac{\mathbf{X} - \mu_{\mathbf{X}}}{\sigma_{\mathbf{X}}}, \quad \bar{\mathbf{Y}} = \frac{\mathbf{Y} - \mu_{\mathbf{Y}}}{\sigma_{\mathbf{Y}}} \quad (3)$$

- Ninguna muestra de vino recibe una calificación superior a 8 ni inferior a 3⁶⁷. Estos casos pueden considerarse una situación hipotéticamente ideal, o probablemente esos datos sufren un sampling bias.

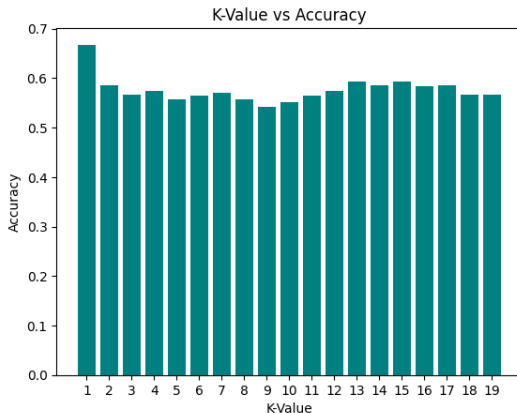


⁶StratifiedKfold

⁷AUC-ROC ó vecinos más cercanos con más influencia usando la inversa de la distancia

- **Paso 1:** Guardar el dataset en la memoria (**equivalente al entrenamiento**).
Dividir el dataset en training y test set.
- **Paso 2:** Inferencia en el algoritmo KNN: el proceso de cálculo de la distancia es un proceso iterativo en el que calculamos la distancia euclidiana de un punto de datos en los datos de prueba con respecto a cada punto de datos en los datos de entrenamiento.
- **Paso 3:** $d(A, B) = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2 + (z_2 - z_1)^2}$.
- Selecciona los k puntos de datos de entrenamiento más cercanos al punto de datos de prueba, basándose en las distancias calculadas. **La etiqueta con la mayor frecuencia de aparición se asigna como clase objetivo al punto de datos de prueba.**

Impacto del valor de k en KNN



Como podemos ver, el valor de $k = 1$ tiene el mayor accuracy. Pero, para $k = 1$, el modelo será muy sensible a los outliers. Por lo tanto, optaremos por $k = 12$, que tiene el segundo mejor accuracy (0.59) .

► Valor de k bajo:

- Un k bajo (e.g., 1 o 2) hace que el modelo sea muy sensible a los **outliers**.
- Los **outliers** son instancias extremas que no siguen las tendencias generales del conjunto de datos.
- Como resultado, las predicciones del modelo se vuelven inestables.

► Valor de k alto:

- A medida que aumenta k , se observa un aumento inicial en la estabilidad del algoritmo.
- Con más vecinos, el efecto de los outliers disminuye debido al voto mayoritario.
- Sin embargo, al seguir aumentando k , la estabilidad comienza a disminuir y la precisión del modelo se deteriora.

- ▶ KNN es costoso computacionalmente.
- ▶ Conforme la dimensionalidad y la escala del dataset aumenta, el tamaño del modelo aumenta, lo que impacta el rendimiento y complejidad del modelo.
- ▶ KNN es sensible a los outliers por lo que pequeños cambios en los datos de entrada (noise) pueden afectar el rendimiento del modelo.

- ▶ Algoritmo de **aprendizaje supervisado** para problemas de **regresión**.
- ▶ Predice el valor de una muestra basándose en los valores de sus k vecinos más cercanos.

Pasos para KNN Regressor

- ▶ Se calcula la distancia entre el punto de interés y los puntos del conjunto de entrenamiento.
- ▶ Se seleccionan los k puntos más cercanos.
- ▶ El **valor predicho** es la **media** (o una combinación) de los valores de esos k vecinos más cercanos.
 - Un k pequeño puede captar más **ruido**.
 - Un k grande puede hacer la predicción demasiado **general**.
- ▶ La distancia **euclidiana** es la más común, pero también se pueden usar otras como **Manhattan** o **Minkowski**.



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

K-nearest Neighbors and K-means Algorithms

EIN087B - Ciencia de Datos
Paralelo 701

Profesor: Jorge Portilla G.
`jorge.portilla@usm.cl`

Departamento de Electrónica e Informática
Universidad Técnica Federico Santa María
Concepción, Chile